

# Tuyển Tập Bài Tập: Quy Hoạch Động Chữ Số (Digit DP)

(16 Bài toán Tuyển chọn Phân cấp từ CSES, Codeforces, AtCoder, SPOJ, POJ & Luogu)

**Tác giả:** Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)  
**Soạn thảo:** Được hỗ trợ và soạn thảo bởi Agentic AI tool  
**Bản quyền** © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

## Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

| STT | Mã Bài / Tên Bài                                   | Nguồn / Nền tảng                         | Cấp độ    | Kỹ thuật Phối hợp                                                | Độ phức tạp Mục tiêu                 |
|-----|----------------------------------------------------|------------------------------------------|-----------|------------------------------------------------------------------|--------------------------------------|
| 1   | <a href="#">CSES 2220 (Counting Numbers)</a>       | <a href="#">CSES Problemset</a>          | Khởi động | Không có 2 chữ số kề nhau trùng nhau + Leading Zeros             | $O(\log_{10} B)$                     |
| 2   | <a href="#">AtCoder DP Task S (Digit Sum)</a>      | <a href="#">Educational DP Contest</a>   | Khởi động | Tổng chữ số chia hết cho $D$ với số lớn $10^{10000}$             | $O(\ K\  \cdot D)$                   |
| 3   | <a href="#">Codeforces 1036C (Classy Numbers)</a>  | <a href="#">Educational Round 50</a>     | Khởi động | Tối đa 3 chữ số khác 0 + Tái sử dụng Memo qua $10^4$ test        | $O(T + \log_{10} R)$                 |
| 4   | <a href="#">SPOJ PR003004 (Digit Sum)</a>          | <a href="#">SPOJ</a>                     | Khởi động | Tính tổng các chữ số của tất cả các số trong $[A, B]$            | $O(\log_{10} B)$                     |
| 5   | <a href="#">Codeforces 628D (Magic Numbers)</a>    | <a href="#">Educational Round 8</a>      | Trung cấp | Số chẵn/lẻ cố định chữ số $d$ & chia hết cho $m$                 | $O(\ B\  \cdot m)$                   |
| 6   | <a href="#">SPOJ BALNUM (Balanced Numbers)</a>     | <a href="#">SPOJ</a>                     | Trung cấp | Trạng thái 3 mức: Chữ số chẵn lẻ lần, lẻ chẵn lần                | $O(\log_{10} B \cdot 3^{10})$        |
| 7   | <a href="#">Codeforces 55D (Beautiful Numbers)</a> | <a href="#">Codeforces Beta Round 51</a> | Trung cấp | Chia hết cho các chữ số + Rời rạc hóa BCNN (2520)                | $O(\log_{10} R \cdot 2520 \cdot 48)$ |
| 8   | <a href="#">SPOJ LUCIFER (Lucifer Number)</a>      | <a href="#">SPOJ</a>                     | Trung cấp | Hiệu chữ số vị trí chẵn - vị trí lẻ là số nguyên tố              | $O(\log_{10} B \cdot 180)$           |
| 9   | <a href="#">Codeforces 1073E (Segment Sum)</a>     | <a href="#">Educational Round 53</a>     | Nâng cao  | Đóng góp giá trị $\{count, sum\}$ + Bitmask $K$ chữ số khác nhau | $O(\log_{10} R \cdot 2^{10})$        |

| STT | Mã Bài / Tên Bài                                              | Nguồn / Nền tảng                             | Cấp độ     | Kỹ thuật Phối hợp                                            | Độ phức tạp Mục tiêu                 |
|-----|---------------------------------------------------------------|----------------------------------------------|------------|--------------------------------------------------------------|--------------------------------------|
| 10  | <a href="#">AtCoder ABC 208 E (Digit Products)</a>            | <a href="#">AtCoder Beginner Contest 208</a> | Nâng cao   | Tích các chữ số $\leq K$ + Bảng băm các ước $\{2, 3, 5, 7\}$ | $O(\log_{10} N \cdot \text{States})$ |
| 11  | <a href="#">POJ 3252 (Round Numbers)</a>                      | <a href="#">POJ 3252</a>                     | Nâng cao   | Digit DP hệ cơ số 2: Số lượng bit 0 $\geq$ bit 1             | $O(\log_2 B)^2$                      |
| 12  | <a href="#">Codeforces 908G (New Year and Original Order)</a> | <a href="#">Good Bye 2017</a>                | Nâng cao   | Sắp xếp chữ số tăng dần + Phân rã đóng góp 9 lần DP          | $O(9 \cdot \ N\ ^2)$                 |
| 13  | <a href="#">Luogu P4127 ([AHOI2009] 数字之和)</a>                 | <a href="#">AHOI 2009 / Luogu</a>            | Chinh phục | Số chia hết cho chính tổng chữ số của nó                     | $O(162^2 \cdot \log_{10} B)$         |
| 14  | <a href="#">Luogu P3413 ([SAC#1] 萌数)</a>                      | <a href="#">SAC 2017 / Luogu</a>             | Chinh phục | Chứa chuỗi con đối xứng $\geq 2$ + Lưu 2 chữ số liền trước   | $O(\ R\  \cdot 100)$                 |
| 15  | <a href="#">Codeforces 507D (The Brand New Function)</a>      | <a href="#">Codeforces Round 287</a>         | Chinh phục | Suffix Digit DP: Tồn tại hậu tố chia hết cho $k$             | $O(n \cdot k)$                       |
| 16  | <a href="#">Codeforces 214B (Haghani and Divisibility)</a>    | <a href="#">Codeforces Round 131</a>         | Chinh phục | Ghép chữ số thành số lớn nhất chia hết cho 2, 3, 5           | $O(N \log N)$                        |

## 1. Cấp Độ 1: Khởi Động & Nền Tảng (Warm-up & Fundamentals)

### Bài 1: [CSES 2220 - Counting Numbers](#)

- **Nguồn:** [CSES Problemset](#).
- **Giới hạn:**  $0 \leq A \leq B \leq 10^{18}$ , Thời gian: 1.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Đếm số lượng số nguyên  $X \in [A, B]$  sao cho không có hai chữ số liền kề nào trong biểu diễn thập phân của  $X$  là giống nhau.

#### Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Đây là bài toán mẫu mực nhất để rèn luyện cờ `leading_zero`. Các số như 0007 thực chất là số 7, hai số 0 ở đầu không được coi là trùng lặp.
- **Bậc 2 (Thiết kế trạng thái):** Gọi `dfs(pos, tight, leading_zero, prev)`:
  - `pos`: Vị trí đang xét ( $0 \dots \text{len} - 1$ ).
  - `prev`: Chữ số vừa được điền ở vị trí trước đó (khởi tạo  $-1$ ).
- **Bậc 3 (Chuyển trạng thái):**
  - Nếu `leading_zero == true` và chọn  $d = 0$ : Số vẫn chưa bắt đầu, gọi tiếp `dfs(pos + 1, tight && (d == limit), true, -1)`.

- Nếu chọn  $d \neq 0$  hoặc `leading_zero == false`: Bắt buộc  $d \neq prev$ , chuyển sang `dfs(pos + 1, tight && (d == limit), false, d)`.
- Kết quả: `solve(B) - (A > 0 ? solve(A - 1) : 0)`.

## Bài 2: [AtCoder DP Task S - Digit Sum](#)

- **Nguồn:** [Educational DP Contest — Task S](#).
- **Giới hạn:**  $1 \leq K \leq 10^{10000}$ ,  $1 \leq D \leq 100$ , Thời gian: 2.0s.
- **Tóm tắt bài toán:** Cho số nguyên dương  $K$  (được biểu diễn dưới dạng xâu lên tới 10000 chữ số) và số nguyên  $D$ . Đếm số lượng số nguyên  $1 \leq X \leq K$  có tổng các chữ số chia hết cho  $D$ . Kết quả lấy modulo  $10^9 + 7$ .

### Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Độ dài  $K$  lên tới  $10^4$  chữ số, không thể nén thành `long long`, bắt buộc duyệt xâu trực tiếp từ trái sang phải.
- **Bậc 2 (Thiết kế trạng thái):** Trạng thái chỉ cần `dp[pos][sum_mod]`:
- `pos`: Từ 0 đến  $|K| - 1$ .
- `sum_mod`: Tổng các chữ số modulo  $D$  ( $0 \leq sum\_mod < D$ ).
- Không cần cờ `leading_zero` vì chữ số 0 đóng góp 0 vào tổng modulo  $D$ .
- **Bậc 3 (Đáy đệ quy & Trừ số 0):** Tại `pos == |K|`, nếu `sum_mod == 0` trả về 1, ngược lại 0. Sau cùng trừ đi 1 (vì bài toán yêu cầu  $1 \leq X$ , không tính số 0). Đảm bảo kết quả dương qua `(ans - 1 + MOD) % MOD`.

## Bài 3: [Codeforces 1036C - Classy Numbers](#)

- **Nguồn:** [Codeforces Educational Round 50](#).
- **Giới hạn:**  $T \leq 10^4$  test cases,  $1 \leq L \leq R \leq 10^{18}$ , Thời gian: 3.0s.
- **Tóm tắt bài toán:** Một số được gọi là "Classy" nếu nó có không quá 3 chữ số khác 0 trong biểu diễn thập phân. Với mỗi truy vấn  $[L, R]$ , đếm số lượng số Classy.

### Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện & Tối ưu đa truy vấn):**  $T = 10^4$  test cases rất lớn. Nhưng cấu trúc số Classy không phụ thuộc vào  $L, R$ . Mảng memo `dp[20][4]` có thể khởi tạo một lần duy nhất dùng cho toàn bộ  $10^4$  truy vấn!
- **Bậc 2 (Thiết kế trạng thái):** `dfs(pos, tight, non_zero_cnt)`:
- `non_zero_cnt`: Số lượng chữ số khác 0 đã điền ( $0 \leq non\_zero\_cnt \leq 3$ ). Nếu  $> 3$ , cắt nhánh trả về 0.
- **Bậc 3 (Chuyển trạng thái):** Khi điền chữ số  $d$ : số lượng chữ số khác 0 mới là `non_zero_cnt + (d > 0)`.

## Bài 4: [SPOJ PR003004 - Digit Sum](#)

- **Nguồn:** [SPOJ PR003004](#).

- **Giới hạn:**  $T \leq 100, 0 \leq A \leq B \leq 10^{15}$ , Thời gian: 1.0s.
- **Tóm tắt bài toán:** Tính tổng các chữ số của tất cả các số nguyên trong đoạn  $[A, B]$ . (Ví dụ  $[10, 12]$  có tổng là  $(1 + 0) + (1 + 1) + (1 + 2) = 5$ ).

#### Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Khác với đếm số lượng, bài này yêu cầu tính tổng các chữ số.
- **Bậc 2 (Đóng góp của chữ số):** Nếu điền chữ số  $d$  tại vị trí  $pos$ , thì  $d$  sẽ xuất hiện trong mọi số hợp lệ tạo ra từ nhánh con phía sau.
- **Bậc 3 (Kỹ thuật DP cấp):** Hàm đệ quy trả về `{count, sum_digits}`:
  - `count`: Số lượng số hợp lệ tạo được.
  - `sum_digits`: Tổng các chữ số của tất cả các số đó.
- Khi điền chữ số  $d$ : `ans.count += child.count, ans.sum_digits += child.sum_digits + child.count * d`.

## 2. Cấp Độ 2: Trung Cấp & Ràng Buộc Modulo/Bitmask (Intermediate & Modulo/Masking)

### Bài 5: [Codeforces 628D - Magic Numbers](#)

- **Nguồn:** [Codeforces Educational Round 8](#).
- **Giới hạn:**  $a \leq b \leq 10^{2000}, m \leq 2000, 0 \leq d \leq 9$ , Thời gian: 2.0s.
- **Tóm tắt bài toán:** Cho số nguyên  $m$  và chữ số  $d$ . Một số là "Magic" nếu nó chia hết cho  $m$ , các chữ số ở vị trí chẵn (1-indexed từ trái sang phải) đều bằng  $d$ , và các chữ số ở vị trí lẻ đều khác  $d$ . Đếm số lượng số Magic trong  $[a, b] \pmod{10^9 + 7}$ .

#### Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Vì  $a, b$  có cùng số lượng chữ số (đề bài đảm bảo  $|a| = |b|$ ), vị trí chẵn lẻ được xác định cố định theo chỉ số  $pos$  ( $pos = 0$  là vị trí 1,  $pos = 1$  là vị trí 2...).
- **Bậc 2 (Ràng buộc vị trí):**
  - Tại  $pos$  lẻ (chỉ số 1-indexed chẵn): Bắt buộc chỉ được điền chữ số  $d$ . (Nếu  $d > limit$  khi `tight`, nhánh này không có cách đi).
  - Tại  $pos$  chẵn (chỉ số 1-indexed lẻ): Được điền mọi  $c \in [0, limit]$  thỏa mãn  $c \neq d$ .
- **Bậc 3 (Chuyển trạng thái modulo):** Cập nhật số dư: `new_rem = (rem * 10 + c) % m`. Đáy đệ quy kiểm tra `rem == 0`.

### Bài 6: [SPOJ BALNUM - Balanced Numbers](#)

- **Nguồn:** [SPOJ BALNUM](#).
- **Giới hạn:**  $T \leq 100, 1 \leq A \leq B \leq 10^{19}$ , Thời gian: 2.0s.
- **Tóm tắt bài toán:** Một số là "Balanced" nếu: mọi chữ số chẵn  $\{0, 2, 4, 6, 8\}$  xuất hiện một số **lẻ** lần, và mọi chữ số lẻ  $\{1, 3, 5, 7, 9\}$  xuất hiện một số **chẵn** lần (tính trên các chữ số có nghĩa xuất hiện ít nhất 1).

lần). Đếm số lượng số Balanced trong  $[A, B]$ .

### Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Mỗi chữ số từ 0 đến 9 có 3 trạng thái độc lập: (1) Chưa từng xuất hiện; (2) Đã xuất hiện chẵn lần; (3) Đã xuất hiện lẻ lần.
- **Bậc 2 (Mã hóa trạng thái tam phân):** Dùng hệ cơ số 3 để nén trạng thái của 10 chữ số:  $3^{10} = 59049$  trạng thái. Hoặc tách thành 2 bitmask:
- `mask_app` (10 bits): Bit  $i = 1$  nếu chữ số  $i$  đã xuất hiện.
- `mask_odd` (10 bits): Bit  $i = 1$  nếu chữ số  $i$  hiện đang có tần suất lẻ.
- **Bậc 3 (Đáy đệ quy):** Khi đến đáy, với mỗi chữ số  $i \in [0, 9]$  có `mask_app` bật: nếu  $i$  chẵn thì bit trong `mask_odd` phải là 1; nếu  $i$  lẻ thì bit trong `mask_odd` phải là 0.

---

## Bài 7: [Codeforces 55D - Beautiful Numbers](#)

- **Nguồn:** [Codeforces Beta Round 51](#).
- **Giới hạn:**  $T \leq 10, 1 \leq L \leq R \leq 10^{18}$ , Thời gian: 4.0s.
- **Tóm tắt bài toán:** Một số được gọi là "Beautiful" nếu nó chia hết cho tất cả các chữ số khác 0 của chính nó. Đếm số lượng số Beautiful trong đoạn  $[L, R]$ .

### Gợi ý 3 bậc:

- **Bậc 1 (Phân tích số học):** Số  $X$  chia hết cho tất cả các chữ số của nó  $\iff X$  chia hết cho LCM của các chữ số đó. Bội chung nhỏ nhất của tập hợp con các chữ số  $\{1, 2, \dots, 9\}$  là ước của  $\text{LCM}(1, 2, \dots, 9) = 2520$ .
- **Bậc 2 (Thu hẹp không gian trạng thái):**
- Số dư của  $X$  chỉ cần tính modulo 2520 ( $rem \in [0, 2519]$ ).
- Giá trị LCM của các chữ số chỉ có đúng **48 giá trị khác nhau** (là các ước của 2520). Ta rời rạc hóa 48 giá trị này thành nhãn  $0 \dots 47$ .
- **Bậc 3 (Độ phức tạp & Bộ nhớ):**
- Bảng `dp[20][2520][48]`: Kích thước  $20 \times 2520 \times 48 \approx 2.4 \times 10^6$  phần tử  $\approx 19$  MB.
- Tái sử dụng mảng `dp` qua tất cả các test cases. Thời gian chạy  $< 0.5$  giây.

---

## Bài 8: [SPOJ LUCIFER - Lucifer Number](#)

- **Nguồn:** [SPOJ LUCIFER](#).
- **Giới hạn:**  $T \leq 100, 1 \leq A \leq B \leq 10^9$ , Thời gian: 1.0s.
- **Tóm tắt bài toán:** Một số là "Lucifer" nếu hiệu giữa tổng các chữ số ở vị trí chẵn và tổng các chữ số ở vị trí lẻ (tính từ phải sang trái, 1-indexed) là một số nguyên tố  $> 0$ . Đếm số lượng số Lucifer trong  $[A, B]$ .

### Gợi ý 3 bậc:

- **Bậc 1 (Định hướng căn lề):** Vị trí 1-indexed được tính từ phải sang trái. Do đó vị trí của chữ số hiện tại phụ thuộc vào **khoảng cách tới chữ số cuối cùng**:  $idx = len - pos$ .

- **Bậc 2 (Thiết kế trạng thái):**
- `diff = sum_even - sum_odd`.
- Giá trị `diff` có thể âm, thuộc đoạn  $[-90, 90]$ . Ta dịch chuyển gốc tọa độ bằng cách cộng 100:  
`offset_diff = diff + 100`.
- **Bậc 3 (Đáy đệ quy):** Tại đây, kiểm tra nếu  $diff > 0$  và  $diff$  là số nguyên tố thì trả về 1.

### 3. Cấp Độ 3: Nâng Cao & Đóng Góp Giá Trị (Advanced & Summation/Products)

#### Bài 9: [Codeforces 1073E - Segment Sum](#)

- **Nguồn:** [Educational Codeforces Round 53](#).
- **Giới hạn:**  $1 \leq L \leq R \leq 10^{18}, 1 \leq K \leq 10$ , Thời gian: 1.0s.
- **Tóm tắt bài toán:** Tính tổng của tất cả các số trong  $[L, R]$  có biểu diễn thập phân chứa **không quá  $K$  chữ số phân biệt**. Kết quả lấy modulo 998244353.

##### Gợi ý 3 bậc:

- **Bậc 1 (Kỹ thuật tính đóng góp):** Để tính tổng giá trị các số, ta lưu trữ cặp `{count, sum}` tại mỗi trạng thái đệ quy.
- **Bậc 2 (Biểu diễn mặt nạ):** `mask` gồm 10 bits đánh dấu các chữ số đã xuất hiện. Điều kiện hợp lệ tại lá:  
`__builtin_popcount(mask) <= K`.
- **Bậc 3 (Công thức đóng góp lũy thừa 10):**
- Giả sử nhánh con điền chữ số  $d$  tại vị trí  $pos$ . Đóng góp của  $d$  vào mỗi số hợp lệ là  $d \times 10^{\text{len}-1-pos}$ .
- `total_sum = child.sum + child.count * d * 10^(len - 1 - pos)`.

#### Bài 10: [AtCoder ABC 208 E - Digit Products](#)

- **Nguồn:** [AtCoder Beginner Contest 208](#).
- **Giới hạn:**  $1 \leq N \leq 10^{18}, 1 \leq K \leq 10^9$ , Thời gian: 2.0s.
- **Tóm tắt bài toán:** Đếm số lượng số nguyên dương  $X \leq N$  sao cho tích các chữ số của  $X$  không vượt quá  $K$ .

##### Gợi ý 3 bậc:

- **Bậc 1 (Nhận xét tập ước số):** Tích các chữ số từ 0 đến 9 chỉ chứa các thừa số nguyên tố 2, 3, 5, 7. Với  $X \leq 10^{18}$ , số lượng tích khác nhau sinh ra từ các chữ số  $\leq 9$  thực tế **chưa tới 50,000 giá trị!**
- **Bậc 2 (Cấu trúc dữ liệu ghi nhớ):** Vì không thể dùng mảng tĩnh `dp[20][10^9]`, ta dùng mảng `unordered_map<long long, long long> memo[20]` để ghi nhớ các tích đã xuất hiện.
- **Bậc 3 (Xử lý chữ số 0):** Cờ `leading_zero` rất quan trọng: trước khi số bắt đầu, số 0 không làm tích bằng 0. Khi số đã bắt đầu, điền số 0 sẽ biến tích thành 0 (luôn  $\leq K$ ).

## Bài 11: [POJ 3252 - Round Numbers](#)

- **Nguồn:** [POJ 3252](#).
- **Giới hạn:**  $1 \leq A < B \leq 2 \times 10^9$ , Thời gian: 1.0s.
- **Tóm tắt bài toán:** Một số là "Round" nếu trong biểu diễn nhị phân của nó, số lượng bit 0 lớn hơn hoặc bằng số lượng bit 1. Đếm số lượng số Round trong  $[A, B]$ .

### Gợi ý 3 bậc:

- **Bậc 1 (Chuyển sang hệ cơ số 2):** Thay vì hệ cơ số 10, chuyển  $B$  thành chuỗi bit nhị phân (tối đa 32 bits).
- **Bậc 2 (Thiết kế trạng thái):** `dfs(pos, tight, leading_zero, cnt0, cnt1)` :
- Chỉ tăng `cnt0` khi `leading_zero == false` và bit điền là 0.
- Khi `leading_zero == true` và bit điền là 0: bỏ qua, không tính vào số bit 0.
- **Bậc 3 (Đáy đệ quy):** Khi `pos == len`, trả về 1 nếu `cnt0 >= cnt1`, ngược lại 0.

## Bài 12: [Codeforces 908G - New Year and Original Order](#)

- **Nguồn:** [Codeforces Good Bye 2017](#).
- **Giới hạn:**  $N \leq 10^{700}$ , Thời gian: 2.0s.
- **Tóm tắt bài toán:** Với mỗi số  $X \in [1, N]$ , sắp xếp các chữ số của nó theo thứ tự không giảm để tạo số  $S(X)$ . Tính tổng  $\sum_{X=1}^N S(X) \pmod{10^9 + 7}$ .

### Gợi ý 3 bậc:

- **Bậc 1 (Biến đổi toán học đỉnh cao):** Một số sau khi xếp tăng dần có thể phân tích thành tổng các chuỗi  $11 \dots 1$ . Cụ thể:

$$S(X) = \sum_{d=1}^9 \underbrace{11 \dots 1}_{\text{số chữ số } \geq d \text{ trong } X}$$

- **Bậc 2 (Tách 9 bài toán con độc lập):** Với mỗi chữ số  $d \in [1, 9]$ , ta chỉ cần đếm: Có bao nhiêu số trong  $[1, N]$  chứa đúng  $c$  chữ số  $\geq d$ ?
- **Bậc 3 (Thực thi):** Mỗi bài toán con là một lượt Digit DP đơn giản với trạng thái `(pos, count_ge_d)`. Tổng độ phức tạp  $O(9 \times |N|^2)$ .

## 4. Cấp Độ 4: Chinh Phục & Ứng Dụng Đỉnh Cao (Mastery & Extreme Applications)

## Bài 13: [Luogu P4127 - 【AHOI2009】同难题 / 数字之和 \(Sum of Digits\)](#)

- **Nguồn:** [AHOI 2009 / Luogu](#).
- **Giới hạn:**  $1 \leq A \leq B \leq 10^{18}$ , Thời gian: 3.0s.

- **Tóm tắt bài toán:** Đếm số lượng số  $X \in [A, B]$  sao cho  $X$  chia hết cho chính tổng các chữ số của  $X$ .

#### Gợi ý 3 bậc:

- **Bậc 1 (Nghịch lý đồng thời):** Nếu đồng thời lưu cả `sum` và `rem % sum`, số trạng thái sẽ bùng nổ vì modulo thay đổi liên tục trong quá trình đệ quy.
- **Bậc 2 (Cố định tổng chữ số mục tiêu):** Với  $B \leq 10^{18}$ , tổng các chữ số tối đa là  $18 \times 9 = 162$ . Ta duyệt qua từng giá trị tổng mục tiêu  $S \in [1, 162]$ !
- **Bậc 3 (Chạy 162 lần Digit DP):** Với mỗi  $S$  cố định:
  - Trạng thái: `dp[pos][current_sum][rem % S]`.
  - Đáy đệ quy: Kiểm tra `current_sum == S && rem == 0`.
  - Tổng thời gian:  $162 \times (19 \times 162 \times 162 \times 10) \approx 8 \times 10^7$  phép tính, chạy trong  $\approx 1.5$  giây.

#### Bài 14: [Luogu P3413 - 【SAC#1】萌数 \(Cute Numbers\)](#)

- **Nguồn:** [SAC 2017 / Luogu](#).
- **Giới hạn:**  $L, R \leq 10^{1000}$ , Thời gian: 1.0s.
- **Tóm tắt bài toán:** Một số là "dễ thương" nếu nó chứa ít nhất một chuỗi con liên tiếp có độ dài  $\geq 2$  là một xâu đối xứng (Palindrome). Đếm số lượng số dễ thương trong  $[L, R] \pmod{10^9 + 7}$ .

#### Gợi ý 3 bậc:

- **Bậc 1 (Bổ đề Palindrome tối thiểu):** Mọi xâu đối xứng độ dài  $\geq 2$  đều phải chứa một tâm đối xứng độ dài 2 (dạng  $aa$ ) hoặc độ dài 3 (dạng  $aba$ ). Do đó, một số chứa xâu đối xứng  $\iff$  Tồn tại 2 chữ số giống nhau cách nhau 0 hoặc 1 vị trí ( $d_i = d_{i-1}$  hoặc  $d_i = d_{i-2}$ ).
- **Bậc 2 (Phương pháp phần bù):** Đếm số các số **KHÔNG** chứa bất kỳ mẫu  $aa$  hay  $aba$  nào, sau đó lấy  $(R - L + 1) - \text{không\_thỏa}$ .
- **Bậc 3 (Thiết kế trạng thái):** `dfs(pos, tight, leading_zero, prev1, prev2)`. Khi điền chữ số  $d$ : bắt buộc  $d \neq \text{prev1}$  và  $d \neq \text{prev2}$ .

#### Bài 15: [Codeforces 507D - The Brand New Function / Bear and Beautiful Numbers](#)

- **Nguồn:** [Codeforces Round 287 \(Div. 2\)](#).
- **Giới hạn:**  $n \leq 1000, k \leq 100, m \leq 10^9$ , Thời gian: 2.0s.
- **Tóm tắt bài toán:** Đếm số lượng số có đúng  $n$  chữ số (không có chữ số 0 ở đầu) sao cho tồn tại ít nhất một hậu tố (suffix) khác 0 chia hết cho  $k$ . Kết quả lấy modulo  $m$ .

#### Gợi ý 3 bậc:

- **Bậc 1 (Đổi chiều duyệt: Suffix DP):** Vì đề bài yêu cầu tính chất trên các **hậu tố**, ta duyệt từ hàng đơn vị sang hàng cao nhất (từ phải sang trái)!
- **Bậc 2 (Thiết kế trạng thái):** `dp[pos][rem][has_divisible_suffix]` :
  - `pos` : Từ 0 đến  $n - 1$ .
  - `rem` : Giá trị của hậu tố hiện tại modulo  $k$ : `new_rem = (rem + d * 10pos) % k`.



- `has_divisible_suffix` : Đã xuất hiện hậu tố nào chia hết cho  $k$  và có giá trị khác 0 chưa.
- **Bậc 3 (Xử lý vị trí đầu tiên  $pos = n - 1$ ):** Chữ số ở vị trí  $n - 1$  (hàng cao nhất) không được bằng 0 ( $d \in [1, 9]$ ).

## Bài 16: [Codeforces 214B - Haghani and Divisibility](#)

- **Nguồn:** [Codeforces Round 131 \(Div. 2\)](#).
- **Giới hạn:**  $N \leq 10^5$ , Thời gian: 2.0s.
- **Tóm tắt bài toán:** Cho tập  $N$  chữ số. Hãy chọn và sắp xếp một tập con các chữ số này để tạo thành số lớn nhất chia hết cho 2, 3 và 5.

### Gợi ý 3 bậc:

- **Bậc 1 (Phân tích điều kiện chia hết):** Số chia hết cho 2, 3 và 5  $\iff$  Chia hết cho 30. Bắt buộc phải có chữ số 0 ở cuối cùng, và tổng các chữ số chia hết cho 3.
- **Bậc 2 (Tham lam bỏ ít chữ số nhất):**
  - Nếu tổng các chữ số chia 3 dư 1: Xóa 1 chữ số nhỏ nhất chia 3 dư 1 (nếu có), hoặc xóa 2 chữ số nhỏ nhất chia 3 dư 2.
  - Nếu tổng các chữ số chia 3 dư 2: Xóa 1 chữ số nhỏ nhất chia 3 dư 2 (nếu có), hoặc xóa 2 chữ số nhỏ nhất chia 3 dư 1.
- **Bậc 3 (Sắp xếp giảm dần):** Sau khi loại bỏ tối thiểu các chữ số, sắp xếp các chữ số còn lại giảm dần và in ra kết quả. Xử lý trường hợp đặc biệt toàn số 0 thì chỉ in duy nhất 0.



### Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.